COLA – Lab de Desenvolvimento Web

Node.js + Express 5 + MongoDB/Mongoose + JWT (cookie) + argon2 + Swagger · React + Vite + Tailwind v4 + axios + React Router — projeto "meu-todo"

1. Ordem de construção (de baixo pra cima, testando cada passo)

#	Passo	Testa com
1	`npm init -y` → instalar pacotes → `"type":"module"` + script `start`	—
2	`index.js` "olá mundo" (express + `express.json()` + `listen`)	navegador localhost:5000
3	`.env` + `config/conn.js` (mongoose.connect)	terminal: "Conectado ao MongoDB"
4	**Model** Tarefa (Schema)	—
5	**Controller + Rotas** do CRUD → `app.use("/tarefas", ...)`	Swagger / Thunder Client
6	Model Usuario + registrar/login/logout (argon2 + JWT + cookie) + cors/cookieParser	Swagger + F12 → Cookies
7	**Middleware** auth + `/me` + `criadaPor` + filtrar tudo por `req.user.id`	2 usuários → 404 na tarefa do outro
8	Swagger (`swagger.js` + `/docs`) — **rodar `npm run swagger` a cada rota nova**	localhost:5000/docs
9	Vite + Tailwind + `.env` (VITE_) + `main.jsx` com BrowserRouter	texto azul na tela
10	`api.js` (axios com `withCredentials`)	"Status 401: Não autenticado." = OK
11	App.jsx (rotas + usuário logado) + Login + Register	login, F5 continua logado, Sair
12	Tarefas.jsx + TarefaForm + TarefaItem (CRUD na tela)	criar/editar/situação/excluir/F5

Por que essa ordem? Rota usa controller, que usa model, que usa a conexão. O front só funciona se a API existir.

Comandos

```
mkdir backend && cd backend
npm init -y
npm i express mongoose dotenv cors cookie-parser
jsonwebtoken argon2 swagger-ui-express
npm i -D nodemon swagger-autogen
npm start            # nodemon index.js
npm run swagger      # gera o JSON do swagger
```

```
npm create vite@latest frontend -- --template react
#  linter: ESLint | TypeScript: No | install now: No
cd frontend
npm install
npm i axios react-router-dom tailwindcss @tailwindcss/vite
npm run dev          # http://localhost:5173
```

Sempre **2 terminais**: backend (`npm start`) e frontend (`npm run dev`). MongoDB tem que estar ligado (Serviços → MongoDB Server).

2. Caminho de uma requisição (o mais importante)

```
[React] clique → handleX → api.js (axios, withCredentials) —HTTP + cookie token→
[index.js] express.json() → cookieParser() → cors() → app.use("/tarefas", tarefaRoutes)
[routes]   router.use(authMiddleware) → router.put("/:id", TarefaController.atualizarTarefa)
[middleware] req.cookies.token → jwt.verify → req.user = {id,nome,email} → next()   (senão 401/403)
[controller] lê req.body / req.params → valida (422) → Model.findOneAndUpdate(...) → res.status(200).json({...})
[model]    Schema valida (required/enum) → MongoDB
[React]    res.data.tarefa → setTarefas(novoArray) → tela re-renderiza
```

Pasta / arquivo	Papel	Pasta / arquivo	Papel
`index.js`	cria app, middlewares globais, swagger, rotas, listen	`.env`	PORT, MONGO_URI, FRONTEND_URL, JWT_SECRET
`config/conn.js`	conecta no Mongo, exporta mongoose	`.gitignore`	node_modules e .env
`models/`	Schemas (o "molde")	front `api.js`	todas as chamadas axios
`controllers/`	lógica (o QUE fazer)	front `App.jsx`	usuário logado + rotas
`routes/`	método+caminho → controller (QUANDO)	front `pages/`	telas (guardam estado)
`middleware/auth.js`	verifica o JWT do cookie	front `components/`	peças reutilizáveis (recebem props)

MVC: Model (dados) · View (React) · Controller (lógica). **REST:** recursos em URLs (`/tarefas`) + método HTTP com significado + JSON, sem estado no servidor (token vai em toda requisição).

3. Backend – código

package.json (o que importa)

```
"type": "module",            ← UMA vez só! (npm init às vezes põe "commonjs")
"scripts": {
  "start": "nodemon index.js",
```

index.js

```
import "dotenv/config";              // 1ª LINHA SEMPRE
import express from "express";
import "./config/conn.js";           // .js obrigatório!
import cookieParser from "cookie-parser";
import cors from "cors";
import swaggerUi from "swagger-ui-express";
```

```
  "swagger": "node swagger.js"
},
```

.env (backend)

```
PORT=5000
MONGO_URI=mongodb://localhost:27017/meu_todo
FRONTEND_URL=http://localhost:5173
JWT_SECRET=minha_chave_super_secreta
```

config/conn.js

```
import mongoose from "mongoose";

const conectar = async () => {
  await mongoose.connect(process.env.MONGO_URI);
  console.log("Conectado ao MongoDB");
};
conectar().catch((error) => console.error("Erro:", error));

export default mongoose;
```

swagger.js

```
import swaggerAutogen from "swagger-autogen";
const doc = {
  info: { title: "Meu To-Do API", description: "Lab Dev Web"
},
  host: "localhost:5000",
};
swaggerAutogen()("./swagger_output.json", ["./index.js"],
doc);
```

models/tarefa.js

```
import mongoose from "../config/conn.js";
const { Schema } = mongoose;

const tarefaSchema = new Schema({
  titulo:     { type: String, required: true },
  descricao:  { type: String, required: true },
  dataLimite: { type: Date,   required: true },
  situacao: {
    type: String,
    enum: ["pendente", "em andamento", "concluida"],
    default: "pendente",
  },
  // relacionamento (≈ chave estrangeira)
  criadaPor: { type: Schema.Types.ObjectId,
               ref: "Usuario", required: true },
}, { timestamps: true });   // createdAt / updatedAt

const Tarefa = mongoose.model("Tarefa", tarefaSchema);
export default Tarefa;      // collection "tarefas"
```

routes/tarefa.routes.js

```
import { Router } from "express";
import TarefaController from
"../controllers/tarefa.controller.js";
import authMiddleware from "../middleware/auth.js";

const router = Router();
router.use(authMiddleware);   // protege TODAS abaixo

router.post("/",      TarefaController.criarTarefa);
```

```
import { createRequire } from "module";
import tarefaRoutes from "./routes/tarefa.routes.js";
import usuarioRoutes from "./routes/usuario.routes.js";

const app = express();

app.use(express.json());   // sem isso req.body = undefined
app.use(cookieParser());   // sem isso req.cookies =
undefined
app.use(cors({
  origin: process.env.FRONTEND_URL, // URL exata, NÃO "*"
  credentials: true,                // permite cookie
}));

// JSON não se importa direto com "type":"module"
const require = createRequire(import.meta.url);
const swaggerFile = require("./swagger_output.json");
app.use("/docs", swaggerUi.serve,
swaggerUi.setup(swaggerFile));

app.use("/tarefas", tarefaRoutes);   // prefixo
app.use("/usuarios", usuarioRoutes);

const PORT = process.env.PORT || 5000;
app.listen(PORT, () =>
  console.log(`Servidor rodando em
http://localhost:${PORT}`));
```

models/usuario.js

```
import mongoose from "../config/conn.js"; // mongoose, 1
"o"!
const { Schema } = mongoose;

const usuarioSchema = new Schema({
  nome:  { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true,
           lowercase: true, trim: true },
  // select:false → não vem no find; pedir com
.select("+senha")
  senha: { type: String, required: true, select: false },
}, { timestamps: true });

const Usuario = mongoose.model("Usuario", usuarioSchema);
export default Usuario;
```

middleware/auth.js

```
import jwt from "jsonwebtoken";

const authMiddleware = (req, res, next) => {
  const token = req.cookies?.token;
  if (!token)
    return res.status(401).json({ message: "Não
autenticado." });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    return next();                    // segue p/ controller
  } catch {
    return res.status(403).json({ message: "Token inválido
ou expirado." });
  }
};
export default authMiddleware;
```

routes/usuario.routes.js

```
import { Router } from "express";
import UsuarioController from
"../controllers/usuario.controller.js";
import authMiddleware from "../middleware/auth.js";

const router = Router();
router.post("/",       UsuarioController.registrar);
router.post("/login",  UsuarioController.login);
router.post("/logout", UsuarioController.logout);
```

```
router.get("/",       TarefaController.listarTarefas);
router.get("/:id",    TarefaController.obterTarefa);
router.put("/:id",    TarefaController.atualizarTarefa);
router.delete("/:id", TarefaController.excluirTarefa);
export default router;
// função SEM () → o Express chama quando chegar a
requisição
```

```
// caminho, middleware, controller (executa nessa ordem)
router.get("/me", authMiddleware, UsuarioController.perfil);
export default router;
```

controllers/tarefa.controller.js (CRUD completo)

```
import Tarefa from "../models/tarefa.js";

export default class TarefaController {
  // POST /tarefas
  static async criarTarefa(req, res) {
    const { titulo, descricao, dataLimite, situacao } = req.body;
    if (!titulo || !descricao || !dataLimite)
      return res.status(422).json({ message: "Campos obrigatórios" });
    try {
      const tarefa = await Tarefa.create({
        titulo, descricao, dataLimite, situacao,
        criadaPor: req.user.id,          // dono = logado
      });
      return res.status(201).json({ message: "Tarefa criada", tarefa });
    } catch (error) {
      return res.status(500).json({ message: "Erro ao criar", error: error.message });
    }
  }

  // GET /tarefas
  static async listarTarefas(req, res) {
    try {
      const tarefas = await Tarefa.find({ criadaPor: req.user.id })
        .populate("criadaPor", "nome email")   // "JOIN"
        .sort({ createdAt: -1 });              // mais novas 1º
      return res.status(200).json({ tarefas });
    } catch (error) {
      return res.status(500).json({ message: "Erro ao listar", error: error.message });
    }
  }

  // GET /tarefas/:id
  static async obterTarefa(req, res) {
    try {                                    // try DENTRO da função!
      const tarefa = await Tarefa.findOne({
        _id: req.params.id, criadaPor: req.user.id });
      if (!tarefa)
        return res.status(404).json({ message: "Tarefa não encontrada" });
      return res.status(200).json({ tarefa });
    } catch (error) {
      if (error.name === "CastError")
        return res.status(400).json({ message: "ID da tarefa inválido" });
      return res.status(500).json({ message: "Erro", error: error.message });
    }
  }
```

```
  // PUT /tarefas/:id
  static async atualizarTarefa(req, res) {
    const { titulo, descricao, dataLimite, situacao } = req.body; // NÃO esquecer
    try {
      const tarefa = await Tarefa.findOneAndUpdate(
        { _id: req.params.id, criadaPor: req.user.id }, // filtro (segurança)
        { titulo, descricao, dataLimite, situacao },     // undefined é ignorado
        { new: true, runValidators: true },              // devolve novo + valida enum
      );
      if (!tarefa)
        return res.status(404).json({ message: "Tarefa não encontrada" });
      return res.status(200).json({ message: "Atualizada", tarefa });
    } catch (error) {
      if (error.name === "CastError")
        return res.status(400).json({ message: "ID da tarefa inválido" });
      if (error.name === "ValidationError")
        return res.status(422).json({ message: "Erro de validação", error: error.message });
      return res.status(500).json({ message: "Erro", error: error.message });
    }
  }

  // DELETE /tarefas/:id
  static async excluirTarefa(req, res) {
    try {
```

```
      const tarefa = await Tarefa.findOneAndDelete({
        _id: req.params.id, criadaPor: req.user.id });
      if (!tarefa)
        return res.status(404).json({ message: "Tarefa não encontrada" });
      return res.status(200).json({ message: "Tarefa excluída" });
    } catch (error) {
      if (error.name === "CastError")
        return res.status(400).json({ message: "ID da tarefa inválido" });
      return res.status(500).json({ message: "Erro", error: error.message });
    }
  }
}
```

Por que findOne com `criadaPor` e não findById? Com findById, qualquer logado que souber o ID edita/apaga a tarefa dos outros. Com o filtro, se não for sua → `null` → 404.

controllers/usuario.controller.js

```
import Usuario from "../models/usuario.js";
import argon2 from "argon2";
import jwt from "jsonwebtoken";

export default class UsuarioController {
  // POST /usuarios
  static async registrar(req, res) {
    const { nome, email, senha } = req.body;
    if (!nome || !email || !senha)
      return res.status(422).json({ message: "Todos os campos são obrigatórios." });
    try {
      const existe = await Usuario.findOne({ email });
      if (existe)
        return res.status(422).json({ message: "E-mail já cadastrado." });
      const senhaHash = await argon2.hash(senha);   // NUNCA senha pura
      const usuario = await Usuario.create({ nome, email, senha: senhaHash });
      return res.status(201).json({ message: "Cadastrado.",
        usuario: { id: usuario._id, nome: usuario.nome, email: usuario.email } });
    } catch (error) {
      return res.status(500).json({ message: "Erro ao cadastrar.", error: error.message });
    }
  }

  // POST /usuarios/logout
  static async logout(req, res) {
    res.clearCookie("token", { httpOnly: true, secure: false, sameSite: "lax" });
    return res.status(200).json({ message: "Logout realizado." });
  }

  // GET /usuarios/me  (usado no F5 do React)
  static async perfil(req, res) {
    try {
      const usuario = await Usuario.findById(req.user.id); // sem senha
      if (!usuario) return res.status(404).json({ message: "Não encontrado." });
      return res.status(200).json({ usuario });
    } catch (error) {
      return res.status(500).json({ message: "Erro.", error: error.message });
    }
  }
```

```
  // POST /usuarios/login
  static async login(req, res) {
    const { email, senha } = req.body;
    if (!email || !senha)
      return res.status(422).json({ message: "Informe e-mail e senha." });
    try {
      const usuario = await Usuario.findOne({ email }).select("+senha");
      if (!usuario)
        return res.status(400).json({ message: "Credenciais inválidas." });

      // ordem: (HASH, senhaDigitada)
      const senhaOk = await argon2.verify(usuario.senha, senha);
      if (!senhaOk)
        return res.status(400).json({ message: "Credenciais inválidas." });

      const token = jwt.sign(
        { id: usuario._id, nome: usuario.nome, email: usuario.email }, // payload
        process.env.JWT_SECRET,
        { expiresIn: "1h" },
      );
```

```
      res.cookie("token", token, {
        httpOnly: true,       // JS da página não lê (protege de XSS)
        secure: false,        // true só com HTTPS (produção)
        sameSite: "lax",
        maxAge: 60 * 60 * 1000, // 1h em MILISSEGUNDOS
      });

      return res.status(200).json({ message: "Login realizado.",
        usuario: { id: usuario._id, nome: usuario.nome, email: usuario.email } });
    } catch (error) {
      return res.status(500).json({ message: "Erro no login.", error: error.message });
    }
  }
}
```

Mesma mensagem p/ email errado e senha errada → não revela quais emails existem. Login é POST (senha no body, não na URL).

4. Tabelas de referência

Mongoose

Método	Retorna / faz
`Model.create({...})`	cria e salva (= `new M().save()`)
`Model.find(filtro)`	array (vazio se nada)
`Model.findOne(filtro)`	1 doc ou `null`
`Model.findById(id)`	1 doc ou `null`
`findOneAndUpdate(filtro, dados, {new:true, runValidators:true})`	doc atualizado (sem new → o ANTIGO)
`findByIdAndUpdate(id, dados, opções)`	idem por id
`findOneAndDelete(filtro)` / `findByIdAndDelete(id)`	doc apagado ou null
`.populate("campo", "nome email")`	troca ObjectId pelo doc (ref)
`.sort({ createdAt: -1 })`	-1 desc · 1 asc
`.select("+senha")` / `.select("nome")`	inclui oculto / escolhe campos

Operadores: `$gt` maior · `$lt` menor · `$ne` diferente · `$in` na lista · `$or: [{..},{..}]`

Schema: `required` · `unique` · `default` · `enum` · `trim` · `lowercase` · `select:false` · `ref` · `timestamps`

Erro	Quando	Status
`CastError`	ID não é ObjectId (ex: "abc"; tem 24 hex)	400
`ValidationError`	viola Schema (enum, required)	422
código `11000`	`unique` duplicado	422

HTTP

Método	Uso	Sucesso
GET	ler	200
POST	criar / ações (login)	201 / 200
PUT	atualizar (recurso)	200
PATCH	atualizar parte (ex: só situação)	200
DELETE	excluir	200 / 204

Código	Significado
200 / 201 / 204	OK / Criado / Sem conteúdo
400	Bad Request (ID inválido, credenciais)
401	Não autenticado ("não sei quem você é")
403	Proibido ("sei quem é, mas não pode" / token inválido)
404	Não encontrado
422	Dados inválidos / faltando (validação)
500	Erro no servidor (catch)

Onde vêm os dados	Exemplo
`req.body`	corpo JSON (POST/PUT) — precisa `express.json()`
`req.params.id`	`/tarefas/:id`
`req.query.busca`	`/tarefas?busca=x`
`req.cookies.token`	precisa `cookieParser()`
`req.user`	colocado pelo middleware

Autenticação – teoria

- **Hash (argon2):** irreversível. Cadastro: `argon2.hash(senha)`. Login: `argon2.verify(hash, digitada)` → true/false. Se o banco vazar, as senhas não aparecem.
- **JWT** = `header.payload.assinatura`. Payload é só Base64 → **legível, NÃO criptografado** (nunca pôr senha). A assinatura (JWT_SECRET) garante que ninguém alterou; alterou → `jwt.verify` lança erro. `iat` = criado em, `exp` = expira em.
- **Cookie httpOnly:** navegador guarda e envia sozinho em toda requisição; JS da página não lê → protege de XSS. Cookie vale pro `localhost` em qualquer porta.
- **CORS:** 5173 e 5000 são origens diferentes → navegador bloqueia se o back não liberar. Precisa **dos dois lados**: back `cors({ origin: URL_exata, credentials: true })` + front `withCredentials: true`.
- **Middleware:** função `(req, res, next)` entre a requisição e o controller. `next()` segue; `res.status().json()` para. `express.json()`, `cors()`, `cookieParser()` também são middlewares.
- **.env:** segredos fora do código. Back: `process.env.X`. Front (Vite): `import.meta.env.VITE_X` (prefixo **VITE_** obrigatório).

5. Frontend – código

vite.config.js

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
import tailwindcss from "@tailwindcss/vite";

export default defineConfig({
  plugins: [react(), tailwindcss()],
});
```

src/index.css · frontend/.env

```
@import "tailwindcss";            /* v4: só isso, sem tailwind.config */

VITE_BASE_URL=http://localhost:5000   # reiniciar npm run dev após mudar
```

src/main.jsx

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import "./index.css";
import App from "./App.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <BrowserRouter>   {/* EM VOLTA do App (Routes/useNavigate) */}
      <App />
    </BrowserRouter>
  </StrictMode>,
);
// StrictMode roda useEffect 2x só em dev (requisição duplicada é normal)
```

src/api.js

```
import axios from "axios";

const api = axios.create({
  baseURL: import.meta.env.VITE_BASE_URL,
  withCredentials: true,   // MANDA O COOKIE (sem isso: sempre 401)
});

export const registrar = (dados) => api.post("/usuarios", dados);
export const login     = (dados) => api.post("/usuarios/login", dados);
export const logout    = () => api.post("/usuarios/logout");
export const getPerfil = () => api.get("/usuarios/me");

export const listarTarefas   = () => api.get("/tarefas");
export const criarTarefa     = (t) => api.post("/tarefas", t);
export const atualizarTarefa = (id, d) => api.put(`/tarefas/${id}`, d);
export const excluirTarefa   = (id) => api.delete(`/tarefas/${id}`);
// sucesso: res.data · erro: error.response?.data?.message
// 2º parâmetro do post/put vira req.body no back
```

src/App.jsx

```
import { useEffect, useState } from "react";
import { Routes, Route, Navigate, useNavigate } from "react-router-dom";
import { getPerfil, logout } from "./api";
import Login from "./pages/Login";
import Register from "./pages/Register";
import Tarefas from "./pages/Tarefas";

export default function App() {
  const [usuario, setUsuario] = useState(null);     // null = ninguém
  const [carregando, setCarregando] = useState(true);
  const navigate = useNavigate();

  useEffect(() => {          // [] = roda 1x (abrir / F5)
    getPerfil()
      .then((res) => setUsuario(res.data.usuario))  // 200 → logado
      .catch(() => setUsuario(null))                // 401 → não
      .finally(() => setCarregando(false));         // sempre
  }, []);

  const handleLogout = async () => {
    await logout(); setUsuario(null); navigate("/login");
```

```
  };

  if (carregando) return <p>Carregando...</p>;

  return (
    <div className="min-h-screen bg-gray-50">
      <header> ... {usuario && (
          <><span>Olá, {usuario.nome}</span>
            <button onClick={handleLogout}>Sair</button></>
      )} </header>
      <main className="max-w-3xl mx-auto p-4">
        <Routes>
          <Route path="/login" element={usuario
            ? <Navigate to="/" />
            : <Login onLogin={setUsuario} />} />
          <Route path="/register" element={usuario
            ? <Navigate to="/" /> : <Register />} />
          <Route path="/" element={usuario      {/* rota protegida */}
            ? <Tarefas /> : <Navigate to="/login" />} />
          <Route path="*" element={<Navigate to="/" />} />
        </Routes>
      </main>
    </div>
  );
}
```

Por que o /me? No F5 o estado do React zera, mas o cookie fica. O App pergunta ao /me: 200 = logado, 401 = vai pro login. **carregando** evita a tela de login "piscar".

src/pages/Login.jsx (estados separados)

```
export default function Login({ onLogin }) {        // props
  const [email, setEmail] = useState("");
  const [senha, setSenha] = useState("");
  const [erro, setErro] = useState("");

  const handleSubmit = async (e) => {
    e.preventDefault();          // form NÃO recarrega a página
    setErro("");
    try {
      const res = await login({ email, senha });
      onLogin(res.data.usuario); // avisa o pai → App redireciona
    } catch (error) {
      setErro(error.response?.data?.message || "Erro ao entrar.");
    }
  };

  return (
    <form onSubmit={handleSubmit} className="space-y-3">
      <input type="email" required value={email}
        onChange={(e) => setEmail(e.target.value)} />  {/* controlado */}
      <input type="password" required value={senha}
        onChange={(e) => setSenha(e.target.value)} />
      {erro && <p className="text-red-600">{erro}</p>}
      <button>Entrar</button>        {/* em form = submit */}
      <Link to="/register">Cadastre-se</Link> {/* sem recarregar */}
    </form>
  );
}
```

src/pages/Register.jsx (um estado-objeto)

```
const [form, setForm] = useState({ nome: "", email: "", senha: "" });
const navigate = useNavigate();

const handleChange = (e) => {
  // copia o resto (...form) e troca o campo pelo "name" do input
  setForm({ ...form, [e.target.name]: e.target.value });
};

const handleSubmit = async (e) => {
  e.preventDefault();
  try {
    await registrar(form);               // { nome, email, senha }
    alert("Cadastro realizado! Faça login.");
    navigate("/login");                  // navegar pelo código
  } catch (error) {
    setErro(error.response?.data?.message || "Erro ao cadastrar.");
  }
```

```
};

<input name="nome"  value={form.nome}  onChange={handleChange} />
<input name="email" value={form.email} onChange={handleChange} />
<input name="senha" type="password" value={form.senha}
       onChange={handleChange} />
// name do input = nome do campo no estado = nome no req.body
```

src/pages/Tarefas.jsx (guarda o estado + API)

```
const [tarefas, setTarefas] = useState([]);
const [editando, setEditando] = useState(null);

useEffect(() => {                                  // READ
  listarTarefas().then((res) => setTarefas(res.data.tarefas))
    .catch((error) => setErro(error.response?.data?.message));
}, []);

const handleSalvar = async (dados) => {            // CREATE / UPDATE
  try {
    if (editando) {
      const res = await atualizarTarefa(editando._id, dados);
      setTarefas(tarefas.map((t) =>
        t._id === editando._id ? res.data.tarefa : t));
      setEditando(null);
    } else {
      const res = await criarTarefa(dados);
      setTarefas([res.data.tarefa, ...tarefas]);
    }
  } catch (error) { alert(error.response?.data?.message); }
};

const handleMudarSituacao = async (tarefa, situacao) => {
  const res = await atualizarTarefa(tarefa._id, { situacao });
  setTarefas(tarefas.map((t) => t._id === tarefa._id ? res.data.tarefa : t));
};

const handleExcluir = async (id) => {                // DELETE
  if (!confirm("Excluir esta tarefa?")) return;
  await excluirTarefa(id);
  setTarefas(tarefas.filter((t) => t._id !== id));
};

return (<>
  <TarefaForm key={editando?._id || "nova"} tarefaInicial={editando}
    onSalvar={handleSalvar} onCancelar={() => setEditando(null)} />
  {tarefas.length === 0 ? <p>Nenhuma tarefa ainda.</p> :
    tarefas.map((t) => (
      <TarefaItem key={t._id} tarefa={t}
        onEditar={() => setEditando(t)}
        onExcluir={() => handleExcluir(t._id)}
        onMudarSituacao={(s) => handleMudarSituacao(t, s)} />
    ))}
</>);
```

src/components/TarefaForm.jsx (cria E edita)

```
export default function TarefaForm({ tarefaInicial, onSalvar, onCancelar }) {
  const [titulo, setTitulo] = useState(tarefaInicial?.titulo || "");
  const [descricao, setDescricao] = useState(tarefaInicial?.descricao || "");
  // input date quer "AAAA-MM-DD" → corta "2026-10-21T00:00:00.000Z"
  const [dataLimite, setDataLimite] =
    useState(tarefaInicial?.dataLimite?.slice(0, 10) || "");

  const handleSubmit = (e) => {
    e.preventDefault();
    onSalvar({ titulo, descricao, dataLimite }); // pai decide criar/editar
    if (!tarefaInicial) { setTitulo(""); setDescricao(""); setDataLimite(""); }
  };
  // ... inputs controlados (required) + <input type="date">
  // {tarefaInicial ? "Salvar alterações" : "Adicionar"}
  // <button type="button" onClick={onCancelar}>  ← type="button" senão vira submit
}
```

src/components/TarefaItem.jsx (só mostra, avisa o pai)

```
const cores = {
  pendente: "bg-yellow-100 text-yellow-800",
  "em andamento": "bg-blue-100 text-blue-800",   // aspas: tem espaço
  concluida: "bg-green-100 text-green-800",
```

```
};
<span className={`text-xs px-2 rounded-full ${cores[tarefa.situacao]}`}>
Prazo: {new Date(tarefa.dataLimite)
  .toLocaleDateString("pt-BR", { timeZone: "UTC" })}  {/* senão: dia -1 */}
<select value={tarefa.situacao}
  onChange={(e) => onMudarSituacao(e.target.value)}>
  <option value="pendente">Pendente</option> ...
<button onClick={onEditar}>Editar</button>
<button onClick={onExcluir}>Excluir</button>
```

key no form: o `useState(inicial)` só usa o valor na 1ª vez. Mudando a `key` o React recria o form com os dados da tarefa clicada.

6. React – resumo dos conceitos

Conceito	Como
Componente	função que retorna JSX · `className` (não class)
useState	`const [v, setV] = useState(ini)` · setV re-renderiza (let não)
useEffect	`useEffect(() => {...}, [])` · [] = 1x · sem [] + set = **loop infinito**
Props	pai → filho: `<Item tarefa={t} />` · filho → pai: chamando função recebida
Input controlado	`value={x} onChange={(e) => setX(e.target.value)}`
Form	`onSubmit` + `e.preventDefault()`
Condicional	`{erro && <p/>}` · `{ok ? <A/> : <B/>}` (ternário)
Lista	`lista.map(t => <Item key={t._id} />)` · key única (o _id)
Função c/ argumento	`onClick={() => excluir(id)}` · **sem () =>** roda na hora!
Classe dinâmica	`className={`p-2 ${cor}`}`
SPA	1 HTML (div#root); Router troca componentes sem recarregar

Imutabilidade (estado de array/objeto)	Código
Adicionar	`setL([nova, ...lista])`
Editar	`setL(lista.map(t => t._id === id ? nova : t))`
Remover	`setL(lista.filter(t => t._id !== id))`
Objeto	`setForm({ ...form, [e.target.name]: e.target.value })`

Nunca `form.email = x` ou `lista.push(x)`: o React não percebe → tela não atualiza.

React Router	Uso
`<BrowserRouter>`	no main.jsx, em volta do App
`<Routes><Route path element />`	path → componente
`<Navigate to="/login" />`	redireciona (rota protegida)
`<Link to="/x">`	clique sem recarregar (≠ <a href>)
`useNavigate()` → `navigate("/x")`	navegar pelo código
`path="*"`	qualquer URL inexistente

Tailwind: `p-4` padding · `m-4 mt-2 mx-auto` margin · `px-3 py-2` · `flex items-center justify-between gap-4` · `flex-col` · `space-y-3` · `w-full max-w-md` · `bg-blue-600 text-white` · `hover:bg-blue-700` · `rounded-lg border` · `text-sm text-xl font-bold` · `min-h-screen` · `grid grid-cols-2`

7. Erros comuns (inclusive os que aconteceram no meu projeto!)

Sintoma / mensagem	Causa → solução
`Cannot use import statement outside a module`	falta `"type":"module"` ou ele está **duplicado** com `"commonjs"` (vale o último!)
`ERR_MODULE_NOT_FOUND .../middleware/auth.js`	caminho ≠ pasta real (`middlewares` vs `middleware`) ou faltou o `.js`
`Failed to resolve import "../components/..."` (Vite)	pasta no lugar errado (estava em `pages/components`, era `src/components`)
`mongoose is not defined`	erro de digitação no import (`moongoose`)
`usuarioRoutes is not defined`	maiúscula/minúscula: importou `UsuarioRoutes`, usou `usuarioRoutes`
`titulo is not defined` → 500 no PUT	esqueceu `const { titulo, ... } = req.body`
ID inválido dá 500 em vez de 400 (sem erro de sintaxe!)	fechou a função antes do `catch` → virou um método chamado "catch". Estrutura: `static async f(req,res) { try {...} catch (error) {...} }`
Outro usuário consegue editar minha tarefa	usou `findByIdAndUpdate` → usar `findOneAndUpdate({ _id, criadaPor: req.user.id })`
PUT devolve a tarefa antiga	faltou `{ new: true }`
Update aceita situação "xyz"	faltou `runValidators: true`
`req.body` undefined	faltou `app.use(express.json())` (ou não mandou JSON no body)
Sempre 401 mesmo logado	faltou `withCredentials: true` no axios ou `cookieParser()`
Erro "CORS policy" no console (F12)	`FRONTEND_URL` errado / faltou `credentials: true` / origin `"*"` · reiniciar o back
`process.env.X` undefined	`import "dotenv/config"` não é a 1ª linha / .env fora da pasta backend
`import.meta.env.X` undefined	sem prefixo `VITE_` / não reiniciou o `npm run dev`
`ECONNREFUSED 127.0.0.1:27017`	MongoDB desligado (Serviços → MongoDB Server → Iniciar)
Login sempre "credenciais inválidas"	faltou `.select("+senha")` (senha vem undefined)
`Cannot set headers after they are sent`	faltou `return` antes de um `res.status()` → respondeu 2x
Rota nova não aparece no /docs	rodar `npm run swagger` de novo + F5
Página recarrega ao enviar o form	faltou `e.preventDefault()`
Data aparece 1 dia antes	`toLocaleDateString("pt-BR", { timeZone: "UTC" })`
Warning "unique key prop"	faltou `key={t._id}` no `.map`
Tudo apagado/rodando ao abrir a tela	`onClick={excluir(id)}` → usar `onClick={() => excluir(id)}`
Loop infinito de requisições	`useEffect` sem o `[]`
Cancelar envia o formulário	faltou `type="button"`
`npm create vite` deixa pasta vazia	ficou esperando resposta (`y` / escolhas) ou foi cancelado → apagar e rodar de novo
package.json quebra	JSON puro: sem comentário, sem vírgula no último item, aspas duplas

Leia a 1ª linha do erro no terminal: ela diz o QUÊ e ONDE (arquivo:linha). `X is not defined` = digitação/maiúscula. `MODULE_NOT_FOUND` = caminho do import.

8. Perguntas teóricas prováveis

- **Middleware?** Função `(req,res,next)` entre requisição e controller (auth, json, cors). `next()` continua.
- **Hash x criptografia?** Hash não volta. Criptografia volta com a chave. Senha = hash.
- **401 x 403?** 401 não identificado; 403 identificado sem permissão / token inválido.
- **PUT x PATCH?** PUT atualiza o recurso; PATCH só parte (ex: `PATCH /tarefas/:id/situacao`).
- **JWT é seguro p/ guardar senha?** Não: payload é legível (Base64). Assinatura só impede alteração.
- **Por que cookie httpOnly?** JS não lê → protege contra XSS; navegador envia sozinho.
- **CORS?** Política do navegador que bloqueia origens diferentes (porta diferente conta) sem permissão do servidor.
- **SQL x NoSQL?** Mongo: documentos JSON (BSON) em collections, sem esquema fixo; o Mongoose põe schema na aplicação.
- **populate?** "JOIN": troca o ObjectId (ref) pelo documento.
- **async/await?** Banco/HTTP são assíncronos (Promise); await espera sem travar. Sempre com try/catch.
- **SPA?** Uma página; React troca componentes; Router muda URL sem recarregar.
- **REST?** Recursos + métodos HTTP + JSON + stateless.
- **MVC?** Model dados · View interface · Controller lógica · Rotas ligam URL ao controller.
- **Por que .env / .gitignore?** Segredos fora do código; node_modules recria com `npm install`.
- **-D no npm?** Dependência só de desenvolvimento (nodemon, swagger-autogen).